

PSI

Center for Scientific Computing,
Theory and Data

HPC Cluster Concepts

Introduction to Slurm

Marc Caubet Serrabou
Paul Scherrer Institut, 17 Aug 2026

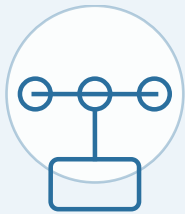
- 1 Introduction
- 2 HPC Cluster Overview
- 3 Slurm Cluster Overview
- 4 Merlin7 Slurm Architecture Overview
 - Small Hands-on: Accessing the Merlin7 cluster
 - Small Hands-on: Understanding hardware topology, NUMA, cores, threads, and affinity on Merlin7 nodes
- 5 Basic Slurm command workflow

Hands-on: Understanding Slurm resource requests and resource use

Goals

The main goal is to provide a practical **introduction to HPC clusters and Slurm**, explaining how computational workloads are submitted and executed, how their progress and resource usage can be monitored, and how to interact effectively with the cluster throughout the lifetime of a job.

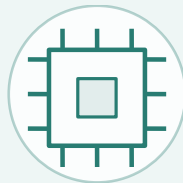
01



Understand resources

Relate nodes, CPUs, tasks, threads and memory to the shape of a workload.

02



Reason about requests

Estimate the CPU and memory capacity a workload needs and understand what that capacity means in practice.

03



Inspect execution

Follow workloads from waiting to running to completion and verify where processes and CPUs actually run.

04

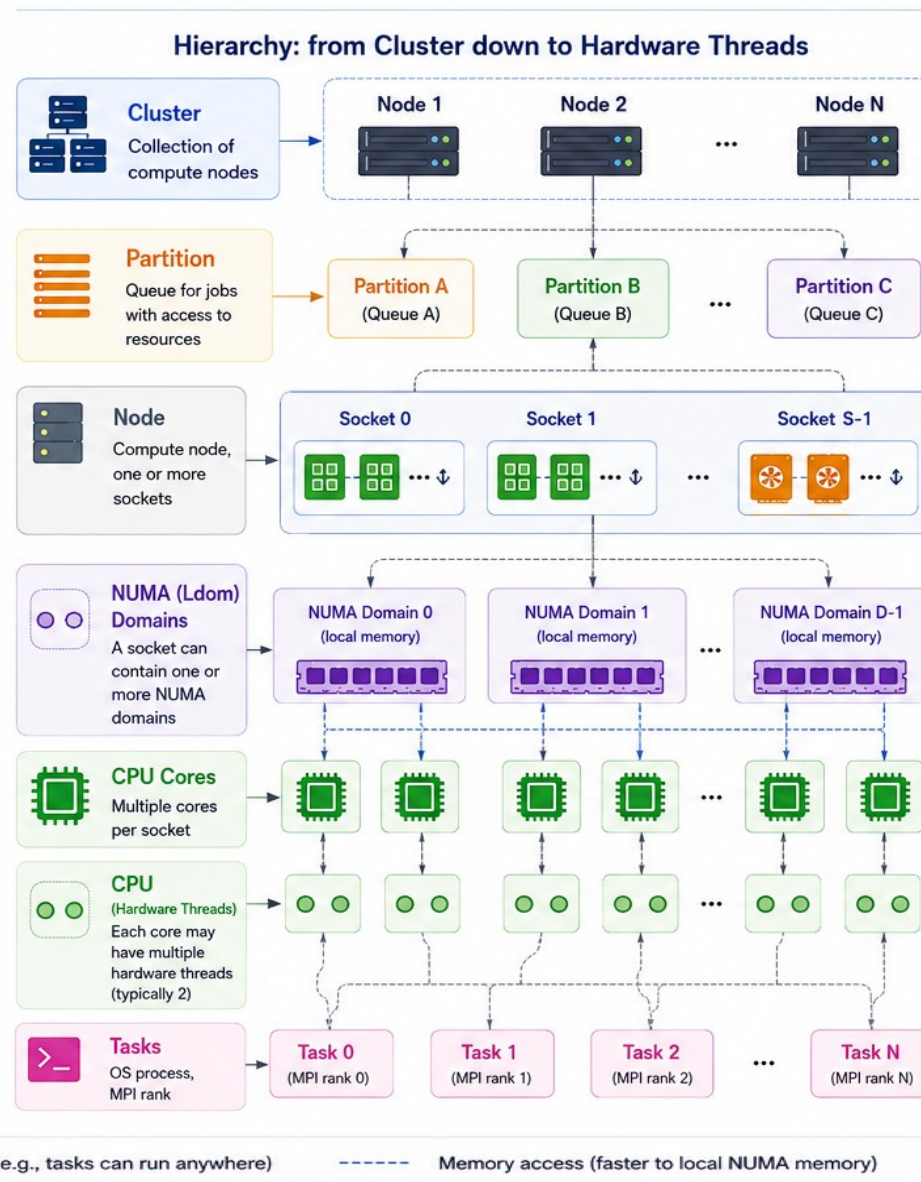


Recognize inefficiency

Spot underuse, oversubscription and inconsistent resource choices—and identify what should be tuned.

HPC Cluster Overview

	Ldom / NUMA	Locality Domain or NUMA (Non-Uniform Memory Access): a set of CPUs that share local memory which they can access faster than memory attached to other domains. A socket can contain one or more NUMA domains (depends on the processor architecture).
	Socket	In general a physical CPU processor chipset. Also known as CPU socket.
	CPU Core	A CPU core is a physical computing unit. 1 CPU core may contain multiple CPU hardware threads (typically 2).
	CPU (Hardware Thread)	A CPU hardware thread. If hyperthreading is not used, this corresponds to a CPU core.
	GPU	Graphics accelerator chipset / processor unit.
	Task	OS process, MPI rank.
	Affinity	The state of being bound to a specific logical processor.
	Partition	Basically a queue for compute jobs with access to certain resources.
	Node	Compute node, which may contain one or more sockets.
	Cluster	Collection of compute nodes.



Key Takeaways

- Tasks run on CPU hardware threads.
- Threads belong to cores, cores belong to sockets, sockets belong to nodes.
- Each NUMA domain has local memory that is faster to access.
- A socket can contain one or more NUMA domains (e.g., AMD EPYC has multiple NUMA domains per socket).
- GPUs are accelerators attached to the node.
- Jobs are submitted to partitions (queues) that provide access to selected resources.
- All nodes together constitute the cluster.

Example (Typical)

Node with 2 × AMD EPYC 7742 64-Core Processor (+ 4 GPUs)

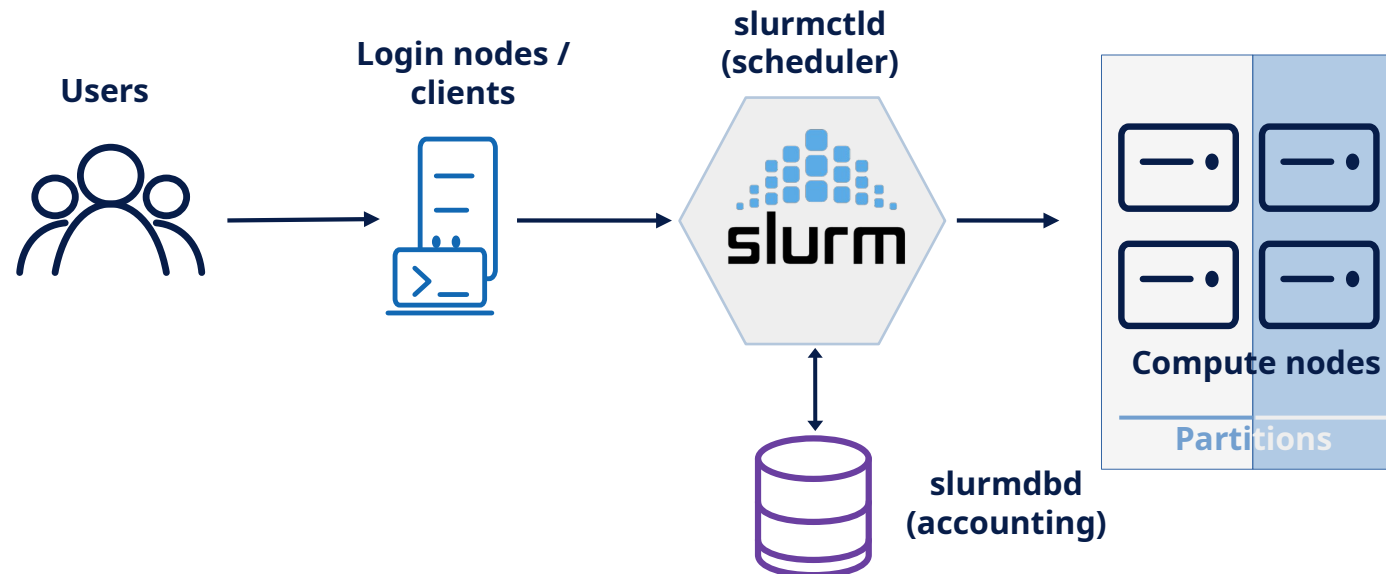
	Socket	2 sockets	
	NUMA Domains	4 per socket (total 8) (EPYC 7742 has 4 NUMA domains per socket)	
	CPU Cores	64 per socket (total 128) 8 cores per CCD × 8 CCDs	
	CPU Threads	128 per socket (total 256) 2 threads per core (SMT)	
	GPUs	4 GPUs per node	

Total per node: 2 sockets | 8 NUMA domains | 128 cores | 256 threads | 4 GPUs

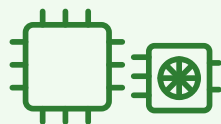
Slurm Cluster Overview

★ Slurm is the workload manager that queues, schedules, launches, and accounts for jobs on an HPC cluster.

-  **1** User submits job
(sbatch, srun, salloc)
-  **2** Slurm schedules resources
-  **3** Job runs on compute nodes,
which are placed in partitions
-  **4** Accounting reports usage



Queues jobs

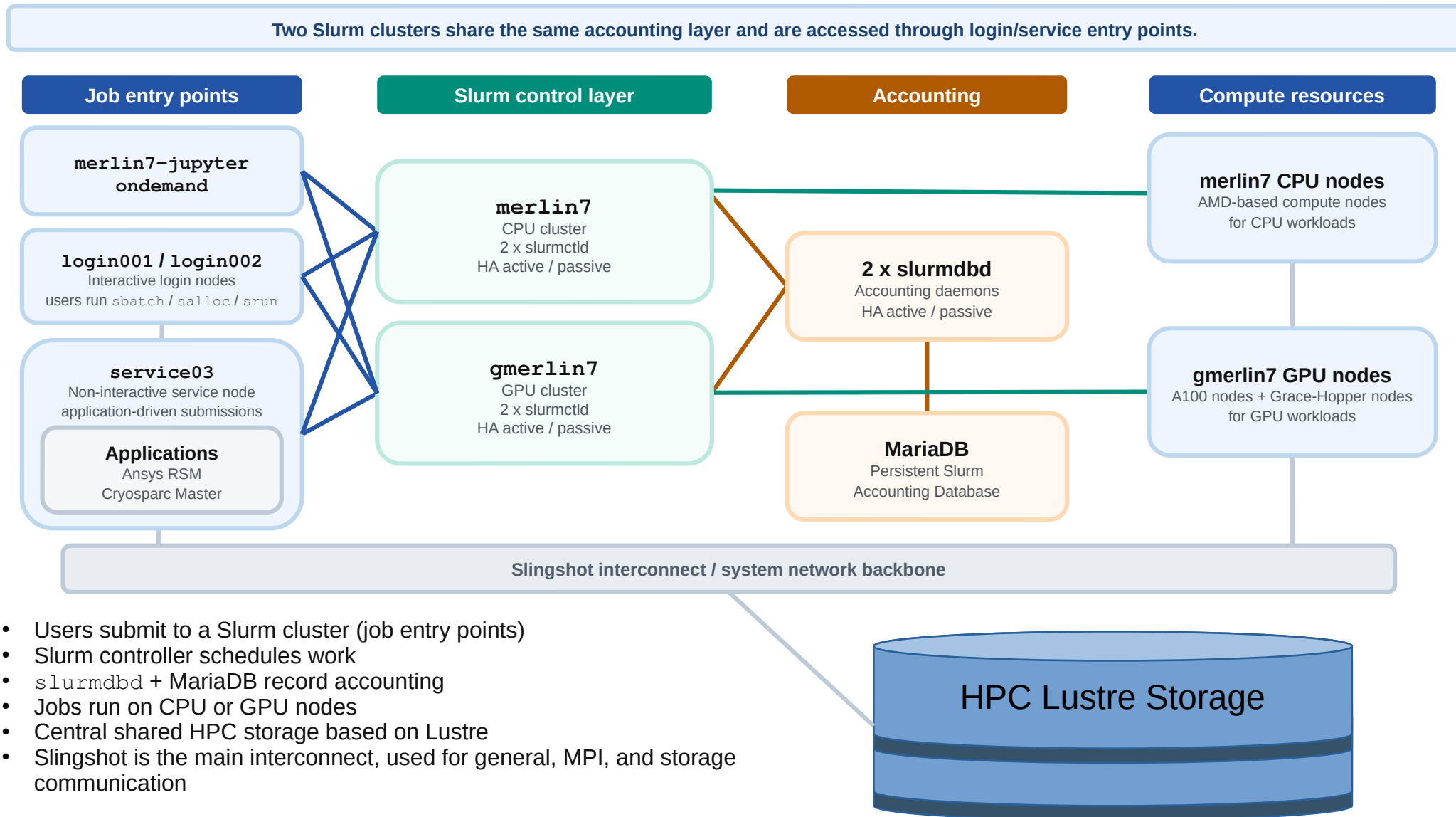


Allocates CPUs,
memory & GPUs



Tracks usage

Merlin7 Slurm Architecture Overview

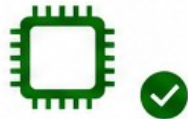


Selector: `SelectType=select/cons_tres`

with `SelectTypeParameters=CR_ONE_TASK_PER_CORE,CR_Core_Memory`

`DefMemPerCPU = 1888`

1. What these parameters do



`CR_ONE_TASK_PER_CORE`

Places one Slurm task per physical core by default (not per hardware thread).

- Avoids unintended core sharing
- Applies to all Slurm tasks: MPI ranks, srun tasks, and other process-style launches



`CR_Core_Memory`

Makes cores and memory consumable resources.

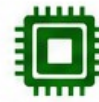
- Slurm allocates cores while also checking the memory requested
- Helps avoid memory overcommit on large nodes



Result: core-aware CPU allocation with memory consumed/accounted alongside the allocation.

2. How it works

Slurm allocates resources as TRES (CPUs, Memory, GPUs/GRES, etc.).



CPUs

...



Memory

...



GPUs (GRES)

Cores and memory are both consumable resources. Slurm selects cores while verifying that the job's memory request (or default memory) fits on the available nodes.

- ✓ Tasks are placed one-per-core by default
- ✓ Memory is accounted together with the cores
- ✓ Reduces risk of memory overcommit

3. Memory default on Merlin

`DefMemPerCPU = 1888`

Default memory per usable allocated CPU (hardware thread / AllocTRES cpu) when no explicit memory is requested (`--mem` or `--mem-per-cpu`).

Implications

- Default memory = $N \times 1888$ MB for N CPUs.
- With `--hint=nomultithread` (HT disabled), only one thread per core is usable for memory calculation. Disabled sibling threads are not included.



Tip: On memory-intensive jobs, set an appropriate `--mem` or `--mem-per-cpu`.

Hands-on: Accessing the Merlin7 cluster



10 min

Choose one access method

SSH

```
login001.merlin7.psi.ch  
login002.merlin7.psi.ch
```

NoMachine

Graphical remote desktop
to the login environment:
`merlin7-nx.psi.ch`

Open OnDemand

Browser-based access:
`ondemand.psi.ch`

```
hostname  
whoami  
pwd
```

Checkpoint

Are you on a login node?
Can you reach your course files?

Use the provided course account: `psicourse-stud[01-50]`

Hands-on: Understanding hardware topology, NUMA, cores, threads, and affinity on Merlin7 nodes



10 min

```
# Login node
lscpu | egrep 'CPU\(s\)|Thread|Core|Socket|NUMA|Model name'
```

```
# CPU compute node
srun -p interactive --reservation psicourse01 \
    bash -lc "lscpu | egrep 'CPU\(s\)|Thread|Core|Socket|NUMA|Model name'"
```

```
# GPU compute node
srun -M gmerlin7 -p a100-hourly --reservation psicourse01 \
    bash -lc "lscpu | egrep 'CPU\(s\)|Thread|Core|Socket|NUMA|Model name'"
```

```
# Optional
numactl -hardware
module load hwloc/2.12.0 ; lstopo
```

Record for each node

- Sockets and physical cores per socket
- Hardware threads per core
- Total visible CPUs
- Number of NUMA domains

Discuss

- Why can nodes with the same CPU model expose different NUMA layouts?
- What topology differences might affect application placement?

Basic Slurm command workflow



1 Submit & launch

sbatch Submit a batch job script
`sbatch job.slurm`

srun Run interactively or launch tasks inside a job
`srun --partition=hourly \
--time=00:30:00 \
--pty bash`

salloc Allocate resources interactively
`salloc --nodes=2 \
--tasks=8 \
--hint=nomultithread \
--time=2:00:00

salloc --ntasks=1 \
--cpus-per-task=6`

2 Monitor

squeue Show running and pending jobs
`squeue -u $USER`

sinfo Show partitions and node states
`sinfo -a`

sprio Show user priority for pending jobs
`sprio -l`

Useful first checks:

- Is my job pending?
- Which partition is available?
- Why is another job ahead of mine?"

3 Control Jobs

scancel Cancel jobs
`scancel <jobid>`

scontrol* Advanced tool to query, manage, and modify jobs, nodes, and partitions in real time
`scontrol show job <jobid>
scontrol hold <jobid>
scontrol release <jobid>
scontrol update JobId=<jobid> TimeLimit=02:00:00`

4 Account & Tune

sacct Show job history and accounting information
`sacct -j <jobid>`

sacctmgr* Inspect accounts, associations, QoS, limits, fairshare, and usage
`sacctmgr show qos \
format=Name%12, Priority, MaxCpuPU, MaxTRES%20

sacctmgr show assoc \
user=$USER \
format=Cluster, Account, MaxSubmit, MaxTRES%20`

seff Display resource utilization and efficiency for completed batch jobs
`seff <jobid>`

***scontrol/*sacctmgr**: Most changes require admin rights.
Users can normally inspect jobs and modify only their own jobs where policy allows.

Hands-on: Understanding Slurm resource requests and resource use



60+ min

Focus

What did the job request?

What did Slurm allocate?

How did `srun` launch work inside the allocation?

How were CPU and memory resources actually used?

The supplied MPI/OpenMP program is an inspection tool and not the learning objective.

Prepare

- Open `mpi_omp_affinity_hands_on/`
- Load the compiler, MPI and hwloc modules
- Compile once on the login node

```
module purge
module load gcc/14.3.0 mpich/5.0.1 hwloc/2.12.0
module list
make clean
make
```

Do not run the placement experiments directly on the login node.